

From code to program

Today we write Riya's canteen bill in C, and learn every piece it needs on the way.

TODAY'S SYLLABUS TOPICS

Eight topics, one program

1 **Structure of a C program**

2 **Life cycle: source code to executable**

3 **Keywords**

4 **Identifiers**

5 **Primitive data types**

6 **Variables**

7 **Constants**

8 **Input/output statements**

Every slide shows its topic number at the top, so you always know where we are.

TODAY'S GOAL

By the end of class, you will write this

A program that asks Riya what she ordered and prints her bill. The same ₹75 bill from Module 1, but this time **you** write the code.

```
≡ IIIT Canteen ≡  
How many chai? 2  
How many samosa? 1  
How many Maggi? 1  
  
Chai x 2 = Rs 20  
Samosa x 1 = Rs 15  
Maggi x 1 = Rs 40  
TOTAL = Rs 75
```

Every C program has the same parts

```
1 #include <stdio.h>

2 /* Riya's canteen bill */
3 int main(void) {
4     printf("Hello, IIIT Canteen!\n");
5     return 0;
6 }
```

Output Hello, IIIT Canteen!



1 Borrow the toolbox

Gives us printf and scanf



2 A note for humans

The compiler ignores comments



3 The front door

The program starts running here



4 One instruction

Every statement ends with ;



5 Job done

Tells the system: finished OK



6 Close the room

Ends main's body { ... }

What happens if...

1 You delete the ; after printf("Hello")?

Compile error: syntax

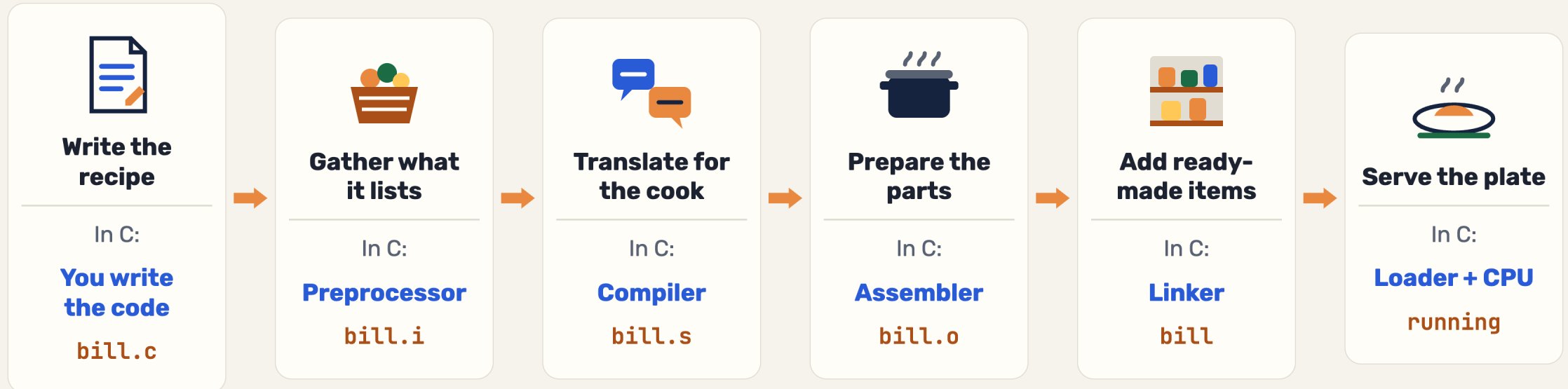
2 You write Main instead of main?

Linker error: C is case-sensitive

3 You delete the comment line?

Nothing: comments are ignored

Like a canteen kitchen turning a recipe into a dish

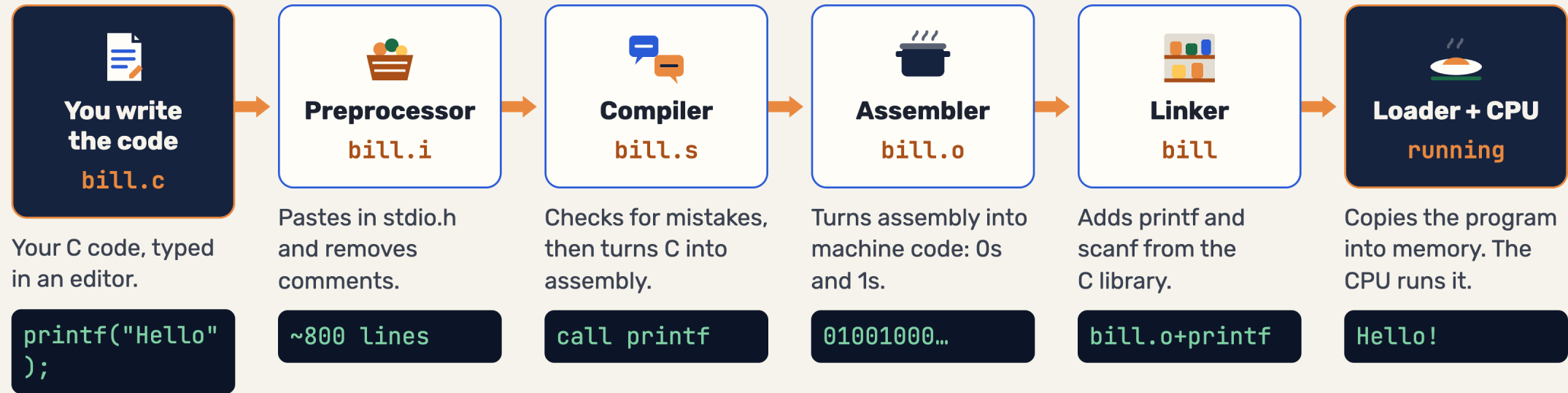


In short

You write C. **gcc** translates it into the computer's language. The computer runs it.

TOPIC 2 OF 8 • LIFE CYCLE: SOURCE CODE TO EXECUTABLE

From bill.c to a running program, step by step



One command does steps 2 to 5: `gcc bill.c -o bill` then `./bill` runs it.

TOPIC 2 OF 8 • LIFE CYCLE: SOURCE CODE TO EXECUTABLE

Watch each stage with gcc

Command	Stops after	You get
<code>gcc -E bill.c -o bill.i</code>	Preprocessor only	bill.i: #include expanded
<code>gcc -S bill.c</code>	Up to the compiler	bill.s: assembly code
<code>gcc -c bill.c</code>	Up to the assembler	bill.o: machine code, not linked
<code>gcc bill.c -o bill</code>	All four stages	bill: the program you run
<code>./bill</code>	Loader runs it	Output on the screen

Try these in the lab terminal. OnlineGDB runs all stages at once when you press Run.

TOPIC 3 OF 8 • KEYWORDS

What is a keyword? A word with a fixed meaning



Road sign: **STOP**

It always means stop. Nobody can put up a STOP sign that means 'go'.



Phone number: **112**

Reserved for emergencies. You can't get 112 as your own mobile number.

In C, keywords work the same way:

int

always means: make a box for a whole number

float

always means: make a box for a decimal number

return

always means: send back a value and finish

So you can't use one as a name:

```
int int = 5;  x  int qty = 5;  ✓
```

C has 32 keywords. These 19 come first



Data types: what kind of box

Today

char	one character, like 'C'
int	a whole number, like 2
float	a decimal number, like 12.5
double	a more precise decimal
void	nothing, no value



Modifiers: change the size or sign

Today, in the lab

short	a smaller whole number
long	a bigger whole number
signed	can be negative (the default)
unsigned	only zero and positive



Decisions: choose what to do

Class 2

if	do this only when it is true
else	otherwise, do this
switch	pick one of many options
case	one option inside a switch
default	when no case matches



Loops: repeat work

Class 3

for	repeat a fixed number of times
while	repeat while something is true
do	run once, then repeat while true
break	stop the loop right now
continue	skip to the next round

Orange keywords are the ones we use today. Blue ones come in the next classes.

The other 13: you'll meet them later



Storage: where a variable lives

Module 3

auto	a normal local variable (default)
register	hint: keep it in a fast CPU register
static	keeps its value between calls
extern	it is declared in another file



Build your own types

Module 5

struct	group data: a student's roll, name, marks
union	different data sharing one space
enum	named values: MON, TUE, WED...
typedef	give a type a new, shorter name



Functions and jumps

Module 3

return	send back a value and finish
goto	jump to a labelled line (avoid it)



Protect a value

const: today

const	this value can never change
volatile	may change outside the program



Measure

Today

sizeof	how many bytes something uses
---------------	-------------------------------

TOPIC 4 OF 8 • IDENTIFIERS

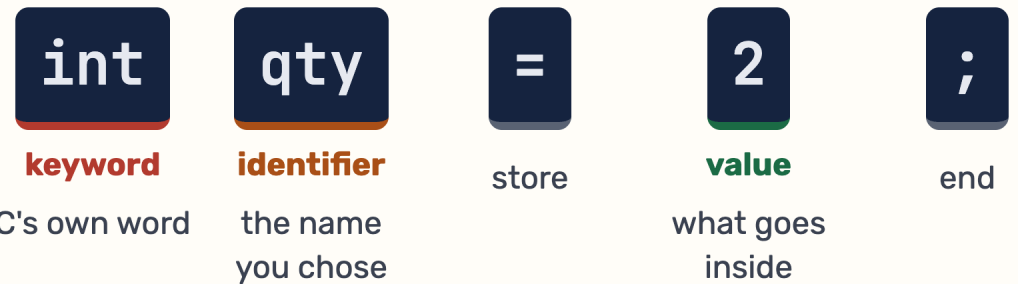
What is an identifier? The name you give to something

In the canteen kitchen



You write the **label**. The label is how you find the jar later.

In C



Identifiers name the things **you** create: variables, constants and functions. Even **main** is an identifier.



Keyword = a word C owns



Identifier = a name you choose

Identifiers: names you choose

The rules

1. Letters, digits and underscore (_) only
2. Must not start with a digit
3. No spaces or symbols like - \$ #
4. Must not be a keyword
5. Case-sensitive: total and Total are different

Valid

chai_price

letters and underscore

total2

digit after the first letter

_count

may start with _ (but avoid)

Invalid

2chai

starts with a digit

total bill

has a space

float

is a keyword

price-1

has a minus sign

TOPIC 4 OF 8 • IDENTIFIERS • VALID OR NOT?

Is this a valid identifier?

1 samosa_qty

Valid

2 3samosa

Invalid: starts with a digit

3 int

Invalid: it is a keyword

4 maggi price

Invalid: has a space

5 Total and total

Both valid, but different names

TOPIC 5 OF 8 • PRIMITIVE DATA TYPES

What is a data type? The kind of container

In the kitchen, you pick the container **before** you fill it: a spoon for salt, a glass for chai, a jug for milk.

In C, the data type is that choice.



Spoon: a pinch of salt



char

1 byte • one character



Glass: one chai



int

4 bytes • a whole number



Jug: measured exactly



double

8 bytes • a precise decimal

A data type tells C three things:

1. What kind of value

a whole number, a decimal or a character

2. How much memory

how many bytes to set aside

3. What you can do

for example, add numbers or compare them

```
int qty = 2; int tells C: qty holds a whole number and needs 4 bytes.
```

Primitive and non-primitive data types



Primitive

The basic types **built into C**. Each holds **one value**, like one cup holds one chai.

`char`

`int`

`float`

`double`

`void`

We learn these **today**.

Non-primitive

Built **from** primitive types. They can hold **many values**.



Array • Module 4

Many values of one type, in a row. Example: Prices of 5 menu items



String • Module 4

A row of characters. Example: The word "Chai"



Structure • Module 5

Different types packed together. Example: A student: roll, name, marks



Pointer • Module 5

Holds the address of another variable. Example: A slip saying where a jar is kept

TOPIC 5 OF 8 • PRIMITIVE DATA TYPES

Primitive data types: pick the right container

char



1 byte • A single character

Canteen: item code 'C'

%c

int



4 bytes • A whole number

Canteen: quantity: 2

%d

float



**4 bytes • A decimal, about
6–7 digits**

Canteen: price 12.50

%f

double



**8 bytes • A precise
decimal, about 15 digits**

Canteen: GST rate 0.05

%lf

Each orange square is **1 byte** of memory. Sizes shown are typical for modern computers.

TOPIC 5 OF 8 • PRIMITIVE DATA TYPES

The four basic types at a glance

Type	Typical size	Holds	printf / scanf
char	1 byte	−128 to 127, or one character	%c
int	4 bytes	about −2.1 billion to +2.1 billion	%d
float	4 bytes	decimals, about 6–7 digits	%f
double	8 bytes	decimals, about 15–16 digits	%lf

Modifiers like short, long and unsigned change the size or range. You will meet them in the lab.

TOPIC 5 OF 8 • PRIMITIVE DATA TYPES

Ask the computer: sizeof

```
/* Module 2, Class 1: how big is each data type? */
#include <stdio.h>

int main(void) {
    printf("char : %zu byte\n", sizeof(char));
    printf("int : %zu bytes\n", sizeof(int));
    printf("float : %zu bytes\n", sizeof(float));
    printf("double : %zu bytes\n", sizeof(double));
    return 0;
}
```

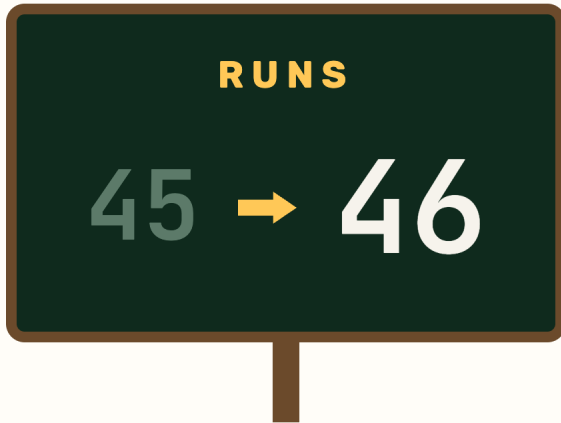
```
char : 1 byte
int : 4 bytes
float : 4 bytes
double : 8 bytes
```

sizeof is a keyword that tells you how many bytes a type uses on **this** computer.

TOPIC 6 OF 8 • VARIABLES

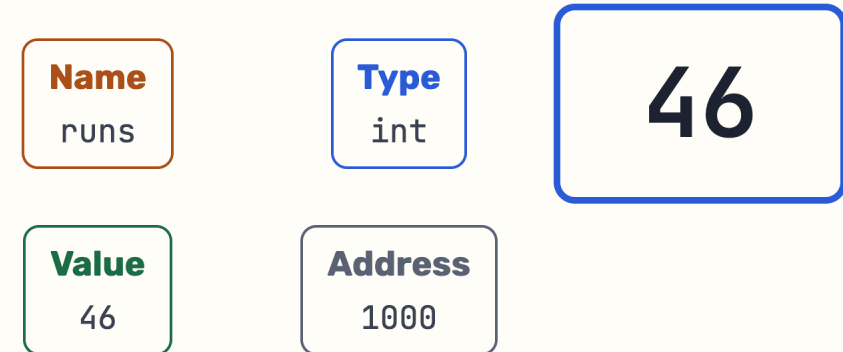
What is a variable? A named box whose value can change

Like a cricket scoreboard



The label **RUNS** stays the same. The number changes every ball. Its value can **vary**, so it's a **variable**.

Every variable has four parts



Name: the identifier. Type: the data type. Value: what's inside.
Address: where it lives in memory.

```
int runs = 45; ➡ runs = 46; Same box, new value.
```

TOPIC 6 OF 8 • VARIABLES

What happens in memory, line by line

```
1 int qty;  
2 qty = 2;  
3 int price = 10;  
4 int total = qty * price;  
5 qty = 3;  
6 total = qty * price;
```

? = garbage: no value stored yet

orange = the box that just changed

– = box not created yet

Line	qty	price	total	What happened
1	?	–	–	qty is created. Value is garbage (?) for now.
2	2	–	–	2 is stored in qty. = means store.
3	2	10	–	price is created with 10 in one line.
4	2	10	20	Reads 2 and 10, stores 20 in total.
5	3	10	20	qty becomes 3. total is STILL 20!
6	3	10	30	Recalculate: $3 \times 10 = 30$.

Variables never update by themselves.

TOPIC 6 OF 8 • VARIABLES

Declare, initialise, assign, use

Declare

```
int qty;
```

Create the box. Do it before you use the variable.

Initialise

```
int qty = 2;
```

Create the box and give it a first value.

Assign

```
qty = 3;
```

Store a new value; the old one is gone.

Use

```
total = qty *  
price;
```

Read the value inside the box.

Good habit: always initialise. An uninitialised variable holds garbage, and the bill comes out wrong.

What does it print?

1 `int a = 5; a = a + 2; a = a * 3; printf("%d", a);`

21

2 `int x = 4, y; y = x; x = 9; printf("%d", y);`

4: y got a copy, not a link

3 `int z; printf("%d", z);`

Garbage: z was never initialised

What is a constant? A value that never changes



Printed price: MRP ₹10

Printed on the packet. The shopkeeper can't change it.



T20 cricket: 20 overs

Fixed by the rules. Runs change; the overs don't.

	Variable	Constant
Example	int runs = 45;	const int OVERS = 20;
Can it change?	Yes, any time	No, never
Real world	The runs on the scoreboard	The overs in a T20 match
Naming habit	small letters: runs	CAPITALS: OVERS

`OVERS = 25;` ➡ The compiler **refuses**: a constant is protected from changes.

Constants: values that must not change

Literal constants

Values written directly in the code:

<code>10</code>	integer
<code>12.5</code>	decimal
<code>'c'</code>	character
<code>"Chai"</code>	string

Named constants

```
#define GST_PERCENT 5  
const int CHAI_PRICE = 10;
```

Try `CHAI_PRICE = 12;` and the compiler refuses: the price is protected.

Why bother? If the chai price changes, you edit **one line**, and nobody can change it by mistake elsewhere.

TOPIC 8 OF 8 • INPUT/OUTPUT STATEMENTS

printf: output to the screen

```
printf("TOTAL = Rs %d\n", total);
```

Text

Printed exactly as written

%d placeholder

Replaced by a value; %d for int

\n escape

Moves to a new line

The value

Fills the placeholder, in order

```
TOTAL = Rs 75
```

Format specifiers and escape sequences

Code	Means	Example	Prints
%d	int	printf("%d", 75)	75
%f	float / double	printf("%f", 12.5)	12.500000
%.2f	2 decimal places	printf("%.2f", 12.5)	12.50
%c	char	printf("%c", 'C')	C
%5d	width 5, right aligned	printf("%5d", 75)	75
\t	tab space	printf("A\tB")	A B

TOPIC 8 OF 8 • INPUT/OUTPUT STATEMENTS

scanf: how a number gets from the keyboard into a variable

Step	Screen	Code	chai	What happens
1	How many chai? _	<code>printf("How many chai? ");</code>	?	printf shows the question on the screen.
2	How many chai? 2 ↵	<code>scanf("%d", &chai);</code>	?	Riya types 2 and presses Enter. scanf waits for this.
3	How many chai? 2 ↵	<code>scanf("%d", &chai);</code>	?	%d tells scanf: read a whole number.
4	How many chai? 2 ↵	<code>scanf("%d", &chai);</code>	?	&chai = ADDRESS of chai (1000): WHERE to put the number.
5	How many chai? 2 ↵	<code>scanf("%d", &chai);</code>	2	scanf stores 2 in the box at address 1000.
6	You ordered 2	<code>printf("You ordered %d", chai);</code>	2	printf reads the box and shows it. No & needed.
Common mistake <code>scanf("%d", chai);</code> No &: scanf gets a garbage address. The program may crash.				

Three scanf mistakes everyone makes once

Forgetting &

Wrong

```
scanf("%d", chai);
```

Right

```
scanf("%d", &chai);
```

Program may crash or store nothing.

Wrong specifier

Wrong

```
float p; scanf("%d", &p);
```

Right

```
scanf("%f", &p);
```

%d for an int, %f for a float.

The leftover Enter

Wrong

```
scanf("%c", &code);
```

Right

```
scanf(" %c", &code);
```

A space before %c skips the Enter left by the previous scanf.

ALL 8 TOPICS TOGETHER

Riya's bill, the whole program

1, 3	1 • Structure: #include, main
4-6	7 • Constants
7	6 • Variables of type int (5)
9-10	8 • Output with printf
11-15	8 • Input with scanf and &
17-18	Calculate (operators: Class 2)
20-23	8 • Formatted output
24	1 • return 0

```
1  #include <stdio.h>
2
3  int main(void) {
4      const int CHAI_PRICE = 10;
5      const int SAMOSA_PRICE = 15;
6      const int MAGGI_PRICE = 40;
7      int chai, samosa, maggi, total;
8
9      printf("≡ IIIT Canteen ≡\n");
10     printf("How many chai? ");
11     scanf("%d", &chai);
12     printf("How many samosa? ");
13     scanf("%d", &samosa);
14     printf("How many Maggi? ");
15     scanf("%d", &maggi);
16
17     total = chai * CHAI_PRICE + samosa * SAMOSA_PRICE
18           + maggi * MAGGI_PRICE;
19
20     printf("\nChai x %d = Rs %d\n", chai, chai * CHAI_PRICE);
21     printf("Samosa x %d = Rs %d\n", samosa, samosa * SAMOSA_PRICE);
22     printf("Maggi x %d = Rs %d\n", maggi, maggi * MAGGI_PRICE);
23     printf("TOTAL = Rs %d\n", total);
24     return 0;
25 }
```

PREDICT BEFORE YOU RUN

What will the program print?

1 Riya enters 2 chai, 1 samosa, 1 Maggi. TOTAL?

Rs 75

2 Arjun enters 3 chai, 0 samosa, 2 Maggi. TOTAL?

Rs 110

3 Which line would you change if chai becomes Rs 12?

Only the const CHAI_PRICE line

RUN IT YOURSELF

Open the program in OnlineGDB

- 1 Open the link or scan the QR code
- 2 Press **Run**
- 3 Type the quantities in the input area at the bottom
- 4 Check: did you get the total you predicted?

onlinegdb.com/r0IkkMjzI



Scan to open bill.c

YOUR TURN • FILL IN THE BLANKS

Complete the program: 5 blanks

```
/* Module 2, Class 1: fill in the 5 blanks marked ____ */
#include <____> /* blank 1 */

int main(void) {
    const int CHAI_PRICE = 10;
    ____ chai; /* blank 2 */
    int total;

    printf("How many chai? ");
    scanf("%d", ____chai); /* blank 3 */

    total = chai * CHAI_PRICE;
    printf("Total = Rs ____\n", total); /* blank 4 */

    return ____; /* blank 5 */
}
```

Each blank tests one topic from today. Fill them in, then run the program to check.

onlinegdb.com/sio4ZG1L6



Scan to open the
blanks program

Work in pairs • 5 minutes

FILL IN THE BLANKS • ANSWERS

Check your answers

1 Blank 1: #include <_____>

stdio.h

2 Blank 2: _____ chai;

int

3 Blank 3: scanf("%d", _____chai);

&

4 Blank 4: printf("Total = Rs _____\n", total);

%d

5 Blank 5: return _____;

0

YOUR TURN • TASK 1 OF 3

Electricity bill



Read how many units a home used. The rate is fixed at Rs 7 per unit. Print the bill.

Rate: Rs 7 (constant)

units: whole number

Algorithm (pseudocode)

```
START
SET RATE ← 7
READ units
bill ← units × RATE
PRINT units, bill
STOP
```

Sample run

```
Units used? 120
Units: 120
Bill = Rs 840
```

Check yourself

- ✓ Which data type for units?
- ✓ Which keyword keeps RATE fixed?
- ✓ Did you put & in scanf?

Now: Write the program in OnlineGDB, then test it with 3 different inputs.

YOUR TURN • TASK 2 OF 3

Report card line



Read a student's roll number and marks in 3 subjects.
Print the roll number and total marks.

4 inputs

all whole numbers

Algorithm (pseudocode)

```
START
READ roll
READ m1, m2, m3
total ← m1 + m2 + m3
PRINT roll, total
STOP
```

Sample run

```
Roll number? 23
Marks in 3 subjects? 85 90 70
Roll 23: total 245 / 300
```

Check yourself

- ✓ One scanf can read three: "%d %d %d"
- ✓ Every variable in scanf needs &
- ✓ Bonus: also print the average. Is it right?

Now: Write the program in OnlineGDB, then test it with 3 different inputs.

YOUR TURN • TASK 3 OF 3

Add tea to Riya's bill



The canteen now sells tea at Rs 12. Add it to the bill program from today.

Start from bill.c

Tea: Rs 12 (constant)

Algorithm (pseudocode)

1. ADD constant TEA_PRICE $\leftarrow$ 12
2. DECLARE variable tea
3. READ tea
4. ADD tea $\times$ TEA_PRICE to total
5. PRINT the tea line

Sample run

```
How many chai? 2
How many samosa? 1
How many Maggi? 1
How many tea? 2
Tea x 2 = Rs 24
TOTAL = Rs 99
```

Check yourself

- ✓ Open bill.c from the Run it slide
- ✓ Change 5 places, not everything
- ✓ Check: $75 + 24 = 99$

Now: Write the program in OnlineGDB, then test it with 3 different inputs.

YOUR TURN • FIX THE BROKEN CODE

This electricity bill has 3 bugs

```
1  /* Electricity bill: find and fix 3 bugs */
2  #include <stdio.h>
3
4  int main(void) {
5      const int RATE = 7
6      int units, bill;
7
8      printf("Units used? ");
9      scanf("%d", units);
10
11     bill = units + RATE;
12
13     printf("Bill = Rs %d\n", bill);
14     return 0;
15 }
```

Find the 3 bugs, fix them, then run it. Each bug is a different type of error.

Line 5: missing ; after 7

Syntax error

Line 9: missing & before units

Runtime: may crash

Line 11: + should be *

Logical: wrong bill



Scan to open

[onlinegdb.com/
rVU0IALIQ](https://onlinegdb.com/rVU0IALIQ)

Tip: one missing ; can cause several error messages. Always fix the **first** error first, then compile again.

TODAY IN ONE SLIDE

Eight topics, one working program

1 Structure `#include, main, statements with ;, return 0`

2 Life cycle `preprocess, compile, assemble, link, load`

3 Keywords `32 reserved words; never use as names`

4 Identifiers `letters, digits, _ ; no digit first; case-sensitive`

5 Data types `char, int, float, double; pick the right size`

6 Variables `named boxes; declare, initialise, assign`

7 Constants `literals, const and #define`

8 Input/output `printf shows values; scanf needs &`

NEXT CLASS

Riya's friends order ₹250. Can the program give them 10% off?

Class 2: operators, type conversion and casting, and decisions with if, else and switch.

Before next class: finish one transfer task and try the practice page.